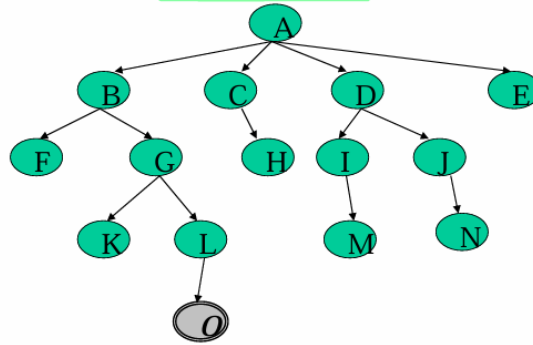


Breadth First Search

■ Application1:

Given the following state space (tree search), give the sequence of visited nodes when using BFS (assume that the node *O* is the goal state):



🔄 BFS Steps (with queue):

Step 0:

Queue: [A]

Visited: []

Step 1:

Visit A → Enqueue children → B, C, D, E

Queue: [B, C, D, E]

Visited: A

Step 2:

Visit B → Enqueue F, G

Queue: [C, D, E, F, G]

Visited: A, B

Step 3:

Visit C → Enqueue H

Queue: [D, E, F, G, H]

Visited: A, B, C

Step 4:

Visit D → Enqueue I, J

Queue: [E, F, G, H, I, J]

Visited: A, B, C, D

Step 5:

Visit E → No children

Queue: [F, G, H, I, J]

Visited: A, B, C, D, E

Step 6:

Visit F → No children

Queue: [G, H, I, J]

Visited: A, B, C, D, E, F

Step 7:

Visit G → Enqueue K, L

Queue: [H, I, J, K, L]

Visited: A, B, C, D, E, F, G

Step 8:

Visit H → No children

Queue: [I, J, K, L]

Visited: A, B, C, D, E, F, G, H

Step 9:

Visit I → Enqueue M

Queue: [J, K, L, M]

Visited: A, B, C, D, E, F, G, H, I

Step 10:

Visit J → Enqueue N

Queue: [K, L, M, N]

Visited: A, B, C, D, E, F, G, H, I, J

Step 11:

Visit K → No children

Queue: [L, M, N]

Visited: A, B, C, D, E, F, G, H, I, J, K

Step 12:

Visit L → Enqueue O

Queue: [M, N, O]

Visited: A, B, C, D, E, F, G, H, I, J, K, L

Step 13:

Visit M → No children

Queue: [N, O]

Visited: A, B, C, D, E, F, G, H, I, J, K, L, M

Step 14:

Visit N → No children

Queue: [O]

Visited: A, B, C, D, E, F, G, H, I, J, K, L, M, N

✅ **Step 15:**

Visit O → GOAL FOUND! 🎯

Visited: A, B, C, D, E, F, G, H, I, J, K, L, M, N, O

🧠 **In Your Diagram:**

After BFS finds O, it **traces back** the path from O to A (its parent, grandparent, etc.).

Let's trace the tree backwards from O:

- O ← child of L
- L ← child of G
- G ← child of B
- B ← child of A

So the **solution path** is:

A → B → G → L → O ✅

***For Lab:

Algorithm (BFS)

1. Start from the source node.
2. Initialize a queue and enqueue the source node.
3. Mark the source node as visited.
4. While the queue is not empty:
 - a. Dequeue a node (current node).
 - b. Process the current node.

- c. For each adjacent (neighbor) node:
 - If it's not visited:
 - i. Mark it as visited
 - ii. Enqueue it

```
#include<bits/stdc++.h>
using namespace std;

void BFS(char start, map<char, vector<char>>&graph, char goal){
    queue<char>q;
    map<char,bool>visited;

    q.push(start);
    visited[start] = true;

    cout<<"BFS traversal: ";

    while(!q.empty()){
        char node = q.front();
        q.pop();

        cout<<node<<" ";

        if(node== goal){
            cout<<"\n Goal node found: "<<goal<<endl;
            return;
        }
        for(char neighbour : graph[node]){
            if(!visited[neighbour]){
                visited[neighbour]= true;
                q.push(neighbour);
            }
        }
    }

    cout<<"\n Goal Node not found! \n";
}

int main(){
    map<char , vector<char>> graph;
    graph['A'] = {'B', 'C', 'D', 'E'};
    graph['B'] = {'F', 'G'};
    graph['C'] = {'H'};
    graph['D'] = {'I', 'J'};
    graph['E'] = {};
    graph['F'] = {};
    graph['G'] = {'K', 'L'};
    graph['H'] = {};
```

```

graph['I'] = {'M'};
graph['J'] = {'N'};
graph['K'] = {};
graph['L'] = {'O'};
graph['M'] = {};
graph['N'] = {};
graph['O'] = {};

char start = 'A';
char goal = 'O';

//if take input from users:

// int edges;
// cout<<"Edges: \n";
// cin>>edges;
// for(int i=0;i<edges;i++){
//     char u,v;
//     cin>>u>>v;
//     graph[u].push_back(v);
// }
// char start, goal;
// cin>>start>>goal;
BFS(start, graph, goal);
}

```

DFS:

```

#include<bits/stdc++.h>
using namespace std;

bool found = false;
void DFSUntil(char node, map<char, vector<char>>&graph, map<char, bool>&visited, char goal){
    if(found){
        return;
    }
    visited[node] = true;
    cout<<node<<" ";

    if(node== goal){
        cout<<"\n goal found: "<<goal<<endl;
        found= true;
    }
}

```

```

        return;
    }
    for( char neighbour: graph[node]){
        if(!visited[neighbour]){
            DFSUntil(neighbour, graph, visited, goal);
        }
    }
}

void DFS(char start, map<char, vector<char>>&graph, char goal){
    map<char,bool>visited;
    cout<<"DFS traversal: ";

    DFSUntil(start, graph, visited , goal);

    if(!found){
        cout<<"\n Goal not found! "<<endl;
    }
}

int main(){
    map<char, vector<char>>graph;
    graph['A'] = {'B', 'C', 'D', 'E'};
    graph['B'] = {'F', 'G'};
    graph['C'] = {'H'};
    graph['D'] = {'I', 'J'};
    graph['E'] = {};
    graph['F'] = {};
    graph['G'] = {'K', 'L'};
    graph['H'] = {};
    graph['I'] = {'M'};
    graph['J'] = {'N'};
    graph['K'] = {};
    graph['L'] = {'O'};
    graph['M'] = {};
    graph['N'] = {};
    graph['O'] = {};

    char start = 'A';
    char goal = 'O';
    DFS(start, graph , goal);
}

```

DLS:

```

#include <bits/stdc++.h>
using namespace std;

bool DLSUtil(char node, map<char, vector<char>>& graph, map<char, bool>& visited,
char goal, int limit) {
    cout << node << " ";

    if (node == goal) {
        cout << "\nGoal found: " << goal << endl;
        return true;
    }

    if (limit <= 0) {
        return false;
    }

    visited[node] = true;

    for (char neighbor : graph[node]) {
        if (!visited[neighbor]) {
            if (DLSUtil(neighbor, graph, visited, goal, limit - 1)) {
                return true;
            }
        }
    }

    return false;
}

void DLS(char start, map<char, vector<char>>& graph, char goal, int limit) {
    map<char, bool> visited;
    cout << "DLS Traversal (limit = " << limit << "): ";

    bool found = DLSUtil(start, graph, visited, goal, limit);

    if (!found)
        cout << "\nGoal not found within depth limit!" << endl;
}

int main() {
    map<char, vector<char>> graph;
    graph['A'] = {'B', 'C', 'D', 'E'};
    graph['B'] = {'F', 'G'};
    graph['C'] = {'H'};
    graph['D'] = {'I', 'J'};
}

```

```

graph['E'] = {};
graph['F'] = {};
graph['G'] = {'K', 'L'};
graph['H'] = {};
graph['I'] = {'M'};
graph['J'] = {'N'};
graph['K'] = {};
graph['L'] = {'O'};
graph['M'] = {};
graph['N'] = {};
graph['O'] = {};

char start = 'A';
char goal = 'O';
int limit = 4;

DLS(start, graph, goal, limit);
}

```

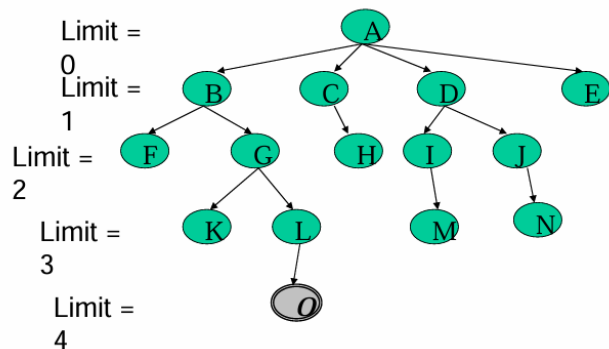
IDS:

A combination of **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**

Iterative Deepening Search (IDS)

Application4:

Given the following state space (tree search), give the sequence of visited nodes when using IDS:



◆ Full IDS Visitation Sequence (Cumulative):

Depth 0: A
 Depth 1: A, B, C, D, E
 Depth 2: A, B, **F**, **G**, C, **H**, D, **I**, **J**, E [node er por e eder child gula oi level er]
 Depth 3: A, B, F, G, **K**, **L**, C, H, D, I, **M**, J, **N**, E
 Depth 4: A, B, F, G, K, L, **O**

```
#include<bits/stdc++.h>
using namespace std;

bool DLSUntil(char node,map<char, vector<char>>&graph ,map<char, bool> &visited,
char goal , int limit ){
    cout<<node<<" ";
    if(node== goal){
        cout<<"GOAL is found! "<<goal<<endl;
        return true;
    }
    if(limit<=0 ){
        return false ;
    }
    visited[node]= true;

    for(char neighbour: graph[node]){
        if(!visited[neighbour]){
            if(DLSUntil(neighbour, graph, visited, goal, limit-1)){
                return true;
            }
        }
    }
}

bool DLS(char start,map<char, vector<char>>&graph , char goal , int limit
){//limit==depth
    map<char, bool> visited;
    cout<<"DLS Traversal Limit"<<limit<<endl;

    return DLSUntil(start, graph, visited , goal, limit);
}

void IDS(char start,map<char, vector<char>>&graph , char goal){
    int depth= 0 ;
    bool found= false;

    while (!found)
    {
        found = DLS(start, graph, goal, depth);
    }
}
```

```

        if(!found){
            cout<<"GOAL not found "<<depth<<endl;
            depth++;
        }
    }
}

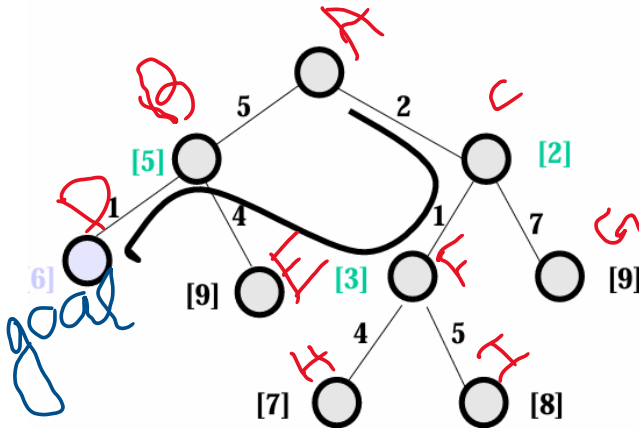
int main(){
    map<char, vector<char>>>graph;
    graph['A'] = {'B', 'C', 'D', 'E'};
    graph['B'] = {'F', 'G'};
    graph['C'] = {'H'};
    graph['D'] = {'I', 'J'};
    graph['E'] = {};
    graph['F'] = {};
    graph['G'] = {'K', 'L'};
    graph['H'] = {};
    graph['I'] = {'M'};
    graph['J'] = {'N'};
    graph['K'] = {};
    graph['L'] = {'O'};
    graph['M'] = {};
    graph['N'] = {};
    graph['O'] = {};

    char start = 'A';
    char goal = 'O';
    IDS(start, graph, goal);
}

```

UCS:

Uniform Cost Search (UCS)



```
#include <bits/stdc++.h>
using namespace std;

typedef pair<char, int> NodeCost;

struct Node {
    char name;
    int cost; // cumulative cost so far
};

// Custom comparator for min-heap
struct CompareNode {
    bool operator()(const Node& a, const Node& b) const {
        return a.cost > b.cost; // smaller cost has higher priority
    }
};

void UCS(map<char, vector<NodeCost>>& graph, char start, char goal) {
    priority_queue<Node, vector<Node>, CompareNode> pq;
    map<char, int> costSoFar; // cost from start to node
    map<char, char> parent;  // to reconstruct path

    costSoFar[start] = 0;
    pq.push({start, 0});

    while (!pq.empty()) {
        Node current = pq.top();
        pq.pop();
    }
}
```

```

        cout << "Visiting Node " << current.name << " (Cost = " << current.cost
<< ")\n";

        // Goal check
        if (current.name == goal) {
            cout << "\nGoal '" << goal << "' found with total cost = " <<
current.cost << endl;

            // Reconstruct path
            vector<char> path;
            for (char n = goal; n != start; n = parent[n])
                path.push_back(n);
            path.push_back(start);
            reverse(path.begin(), path.end());

            cout << "Path: ";
            for (char n : path) cout << n << " ";
            cout << endl;
            return;
        }

        // Explore neighbors
        for (auto& edge : graph[current.name]) {
            char neighbour = edge.first;
            int newCost = current.cost + edge.second;

            // If neighbor not visited OR cheaper path found
            if (!costSoFar.count(neighbour) || newCost < costSoFar[neighbour]) {
                costSoFar[neighbour] = newCost;
                parent[neighbour] = current.name;
                pq.push({neighbour, newCost});
            }
        }
    }

    cout << "Goal not found!\n";
}

int main() {
    map<char, vector<NodeCost>> graph;

    graph['P'] = {{'Q', 1}, {'R', 4}};
    graph['Q'] = {{'R', 2}, {'T', 12}, {'S', 5}};
    graph['R'] = {{'S', 2}};
    graph['S'] = {{'T', 3}};

```

```

    char start = 'P';
    char goal = 'T';

    UCS(graph, start, goal);
    return 0;
}

```

A* search :

```

#include <bits/stdc++.h>
using namespace std;

typedef pair<char, int> NodeCost;

struct Node {
    char name;
    int g; // cost so far
    int f; // total estimated cost
};

// Custom comparator for min-heap
struct CompareNode {
    bool operator()(const Node& a, const Node& b) const {
        return a.f > b.f; // smaller f has higher priority
    }
};

void AStar(map<char, vector<NodeCost>>& graph, map<char, int>& heuristic, char
start, char goal) {
    priority_queue<Node, vector<Node>, CompareNode> pq;
    map<char, int> gCost;
    map<char, char> parent; // for path reconstruction
    gCost[start] = 0;

    pq.push({start, 0, heuristic[start]});

    while (!pq.empty()) {
        Node current = pq.top();
        pq.pop();

        cout << "Visiting Node " << current.name << " (f=" << current.f << ", g="
<< current.g << ")\n";
    }
}

```

```

        if (current.name == goal) {
            cout << "\nGoal '" << goal << "' found with total cost = " <<
current.g << endl;

            // reconstruct path
            cout << "Path: ";
            vector<char> path;
            for (char n = goal; n != start; n = parent[n])
                path.push_back(n);
            path.push_back(start);
            reverse(path.begin(), path.end());
            for (char n : path) cout << n << " ";
            cout << endl;
            return;
        }

        for (auto& edge : graph[current.name]) {
            char neighbour = edge.first;
            int cost = edge.second;

            int newG = current.g + cost;
            int newF = newG + heuristic[neighbour];

            // If neighbour not visited OR found a cheaper path
            if (!gCost.count(neighbour) || newG < gCost[neighbour]) {
                gCost[neighbour] = newG;
                parent[neighbour] = current.name;
                pq.push({neighbour, newG, newF});
            }
        }
    }

    cout << "Goal not found!\n";
}

int main() {
    map<char, vector<NodeCost>> graph;

    graph['P'] = {{'Q', 1}, {'R', 4}};
    graph['Q'] = {{'R', 2}, {'T', 12}, {'S', 5}};
    graph['R'] = {{'S', 2}};
    graph['S'] = {{'T', 3}};

    map<char, int> heuristic = {

```

```

        {'P', 7},
        {'Q', 6},
        {'R', 2},
        {'S', 1},
        {'T', 0}
    };

    char start = 'P';
    char goal = 'T';

    AStar(graph, heuristic, start, goal);
    return 0;
}

```

$f(n) = g(n) + h(n)$

P	7
Q	6
R	2
S	1
T	0

$P \rightarrow Q = 1 + 7 = 8$
 $P \rightarrow R = 4 + 2 = 6$
 $P \rightarrow R \rightarrow S = 4 + 2 + 1 = 7$
 $P \rightarrow Q \rightarrow R = 1 + 2 + 2 = 5$
 $P \rightarrow Q \rightarrow S = 1 + 5 + 1 = 7$
 $P \rightarrow Q \rightarrow T = 1 + 12 + 0 = 13$
 $P \rightarrow Q \rightarrow R \rightarrow S = 1 + 2 + 2 + 1 = 6$

~~$P \rightarrow Q = 7$~~
 ~~$P \rightarrow R = 6$~~
 $P \rightarrow R \rightarrow S = 7$
 ~~$P \rightarrow Q \rightarrow R = 5$~~
 $P \rightarrow Q \rightarrow S = 7$
 $P \rightarrow Q \rightarrow T = 13 \times$
 $P \rightarrow R$

```

Visiting Node P (f=7, g=0)
Visiting Node R (f=6, g=4)
Visiting Node Q (f=7, g=1)
Visiting Node R (f=5, g=3)
Visiting Node S (f=6, g=5)
Visiting Node S (f=7, g=6)
Visiting Node T (f=8, g=8)

```

```

Goal 'T' found with total cost = 8
Path: P Q R S T

```


MIN_MAX:

Minimax is used in **game-playing AI** (like Tic-Tac-Toe, Chess, etc.).

TIC TAC TOE:

RULE BASED:

This approach is just a bunch of **manual rules** — sort of like a human's “common sense” logic.

Basic Rules:

1. If you can **win this turn**, do it.
2. Else if the opponent can **win next turn**, block them.
3. Else if the **center** is free, take it.
4. Else if any **corner** is free, take it.
5. Else take any **side** cell.

That's it.

No recursion, no future simulation — just pattern-based logic.

```
#include <bits/stdc++.h>
using namespace std;

#define AI 'X'
#define HUMAN 'O'
//1,
void printBoard(vector<vector<char>>&b){
    cout<<"\n";

    for(int i=0;i<3;i++){
        for(int j=0;j<3;j++){
            cout<<" "<<b[i][j]<<" ";
            if(j < 2 ) cout<<"|"; //0, 1 column er jonno ei dag ta lagbeee
        }
        cout<<"\n";
        //col shes ,eibar row er jonno:
        if(i<2) cout<<"---|---|---\n"; //row ekto beshi kaj korbeee , 3 ta
        chutor(3 bar) por ekta lomb( total 2 )
    }
    cout<<"\n"; //function declare er por ekta , j er por shes hoile ekta , poro for
    (i er ta) shes hoile ekta , total 3 ta "NEW LINE"
}
//2:
bool isMovesLeft(vector<vector<char>>&board){
```

```

        for(auto& row: board){// Checks if any _ (empty space) is left.
            for(char c: row){//ekbar auto row board er jonno , porer bar char c ashle
row right e
                if(c=='_') return true;
            }
        } return false ;
    }

//3:
bool checkWin(vector<vector<char>>&b, char p ){//player ke donote korar jonno
char p ashbe, AI or HUMAN
    for(int i=0; i<3;i++){
        //row ,col borabor or er por khali swipe hobe
        if((b[i][0]==p && b[i][1]==p && b[i][2]==p) ||
            (b[0][i] == p && b[1][i] == p && b[2][i] == p))
            return true;
        }
//X korle jei 5 ta ase ogula, make cross check [For loop er shes e ]
        if ((b[0][0] == p && b[1][1] == p && b[2][2] == p) ||
            (b[0][2] == p && b[1][1] == p && b[2][0] == p))
            return true;

        return false;
    }

//4. Evaluate curretn board:

int evaluate(vector<vector<char>>&board){
    if(checkWin(board, AI)) return +10;
    if(checkWin(board, HUMAN)) return -10;

    return 0;
}

//6. MINIMAX function
int minimax(vector<vector<char>>&board , int depth , bool isMax){
    int score = evaluate(board);

    if(score ==10 || score== -10){
        return score;
    }
    if(!isMovesLeft(board)){
        return 0;//jodi jkono moves na thake
    }

```

```

    if(isMax){
        int best = -1000;
        for(int i=0; i<3 ;i++){
            for(int j=0; j<3 ;j++){
                if(board[i][j]=='_'){
                    board[i][j] = AI;
                    best = max(best, minimax(board, depth+1, false));
                    board[i][j] = '_';
                }
            }
        }
        return best - depth;
    }
    else {
        int best = 1000;
        for(int i=0; i<3 ;i++){
            for(int j=0; j<3 ;j++){
                if(board[i][j]=='_'){
                    board[i][j] = HUMAN;
                    best = min(best, minimax(board, depth+1, false));
                    board[i][j] = '_';
                }
            }
        }
        return best + depth;
    }
}

```

//7. best move of AI:

```

pair<int,int> findBestMove(vector<vector<char>>&board){
    int bestVal = -1000;

    pair<int,int > bestMove = {-1, -1};

    for(int i=0 ;i<3 ;i++){
        for(int j=0; j<3; j++){
            if(board[i][j] == '_'){
                board[i][j] = AI;
                int moveVal = minimax(board, 0, false);
                board[i][j] = '_';

                if(moveVal > bestVal){
                    bestMove = {i, j };
                    bestVal = moveVal;
                }
            }
        }
    }
    return bestMove;
}

```

```

    }
    }
}

return bestMove;
}

//5.P vs C
void gamePvC_Minimax(){
    vector<vector<char>>board(3, vector<char>(3,'_')); // ei board ei 1, 2, 3, ...
egulay
    cout<<"\n == Playere vs Computer (MInMax AI)===\n";
    printBoard(board); //age board ta print korbe

    while(true){
        //HUMAN PAAAAAART:
        int r,c;
        cout<<"Your move(r,c): \n";

        cin>>r>>c;
        if(r<0 || r>2 || c<0 || c>2 || board[r][c]!= '_'){
            cout<<"INvalid , try again\n";
            continue;
        }

        board[r][c] = HUMAN;
        printBoard(board);

        if(checkWin(board, HUMAN)){ //board, r char p
            cout<<"YOU win\n";
            break;
        }
        if(!isMovesLeft(board)){ //kono move left nai , tao game shes taile draw
            cout<<"IT's Draw! \n";
            break;
        }
        //COMputer PART:
        cout<<"Computer is thinking: \n ";
//er por 6, 7 step :
        pair<int ,int> move = findBestMOve(board);
        board[ move.first][move.second] = AI;
        printBoard(board);

        if(checkWin(board,AI)){

```

```

        cout<<"COMPOUTER WINS \n";
        break;
    }
    if(!isMovesLeft(board)){//kono move left nai , tao game shes taile draw
        cout<<"IT's Draw! \n";
        break;
    }//agertai copy paste
}

}

int main(){
    cout<<"TIC TAC MINIMAX\n \n ";

    gamePvC_Minimax();
    cout<<"====Game OVER!===="<<endl;
    return 0;
}

```

CSP:

```

// CSP:

#include<bits/stdc++.h>
using namespace std;

bool issafe(char node, char color, map<char, vector<char>>&adj,
map<char, char>&assignment)
{
    for(char neighbour: adj[node])
    {
        if(assignment[neighbour]==color)
            return false;
    }
    return true;
}

bool solvecsp(map<char, vector<char>>&adj, map<char, char>&assignment,
vector<char>&colors, vector<char>&nodes, int index)
{
    if(index==nodes.size())
        return true;
}

```

```

    char node= nodes[index];

    for(char color: colors)
    {
        if(issafe(node,color,adj,assignment))
        {
            assignment[node]=color;
            if(solvecsp(adj,assignment,colors,nodes,index+1))
                return true;
            assignment[node]='\0';
        }
    }
    return false;
}

int main()
{
    map<char,vector<char>>adj;
    adj['A']={'B','C'};
    adj['B']={'A','C'};
    adj['C']={'A','B'};

    map<char,char>assignment;
    vector<char>nodes={'A','B','C'};
    vector<char>colors={'R','G','B'};

    if(solvecsp(adj,assignment,colors,nodes,0)){
        cout<<"Solved"<<endl;
        for(char node:nodes)
        {
            cout<<node<<"-->"<<assignment[node]<<endl;
        }
    }
    else
        cout<<"OOPS! I am sorry"<<endl;
}

```

N-Queen:

```

#include <bits/stdc++.h>
using namespace std;

// Check if it is safe to place queen at (row, col)
bool isSafe(vector<int> &board, int row, int col) {

```

```

    for (int i = 0; i < row; i++) {
        if (board[i] == col || abs(board[i] - col) == row - i)
            return false; // same column or diagonal conflict
    }
    return true;
}

// Recursive function to solve N-Queens
bool solveNQueens(vector<int> &board, int row, int N) {
    if (row == N) return true; // All queens placed successfully

    for (int col = 0; col < N; col++) {
        if (isSafe(board, row, col)) {
            board[row] = col; // Place queen
            if (solveNQueens(board, row + 1, N)) // Move to next row
                return true;
            board[row] = -1; // Backtrack
        }
    }
    return false; // No valid column found in this row
}

// Function to print the board
void printBoard(vector<int> &board, int N) {
    cout << "Queen positions (row->column): ";
    for (int i = 0; i < N; i++) {
        cout << board[i] + 1 << " "; // +1 for 1-indexed column
    }
    cout << "\n\nBoard layout:\n";

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i] == j) cout << "Q ";
            else cout << ". ";
        }
        cout << endl;
    }
}

int main() {
    int N = 8; // Change N for NxN board
    vector<int> board(N, -1);

    if (solveNQueens(board, 0, N)) {
        cout << "Solution Found:\n";
    }
}

```

```

        printBoard(board, N);
    } else {
        cout << "No solution exists\n";
    }

    return 0;
}

```

8 puzzle :

```

#include <bits/stdc++.h>
using namespace std;

struct Puzzle {
    vector<vector<int>> board;
    int g; // cost so far
    int f; // total cost = g + h
    string path; // path of moves (optional)
};

// Custom comparator for priority queue
struct ComparePuzzle {
    bool operator()(Puzzle const &a, Puzzle const &b) {
        return a.f > b.f; // smaller f has higher priority
    }
};

// Goal state
vector<vector<int>> goal = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 0}
};

// Manhattan distance heuristic
int manhattan(const vector<vector<int>> &board) {
    int dist = 0;
    for(int i=0;i<3;i++){
        for(int j=0;j<3;j++){
            int val = board[i][j];

```



```

        if(val != 0){
            int goal_row = (val-1)/3;
            int goal_col = (val-1)%3;
            dist += abs(i - goal_row) + abs(j - goal_col);
        }
    }
}

return dist;
}

// Check if current board matches goal
bool isGoal(const vector<vector<int>> &board){
    return board == goal;
}

// Generate new states by moving the blank
vector<pair<vector<vector<int>>, string>> getMoves(const vector<vector<int>>
&board){
    vector<pair<vector<vector<int>>, string>> nextStates;
    int x, y;
    // find blank
    for(int i=0;i<3;i++)
        for(int j=0;j<3;j++)
            if(board[i][j] == 0) { x=i; y=j; break; }

    vector<pair<int,int>> dirs = {{-1,0},{1,0},{0,-1},{0,1}};
    vector<string> moveNames = {"U","D","L","R"};

    for(int k=0;k<4;k++){
        int nx = x + dirs[k].first;
        int ny = y + dirs[k].second;
        if(nx>=0 && nx<3 && ny>=0 && ny<3){
            vector<vector<int>> newBoard = board;
            swap(newBoard[x][y], newBoard[nx][ny]);
            nextStates.push_back({newBoard, moveNames[k]});
        }
    }
    return nextStates;
}

// A* search
void solvePuzzle(vector<vector<int>> start){
    priority_queue<Puzzle, vector<Puzzle>, ComparePuzzle> pq;
    set<vector<vector<int>>> visited;

```

```

pq.push({start, 0, manhattan(start), ""});
visited.insert(start);

while(!pq.empty()){
    Puzzle cur = pq.top(); pq.pop();

    if(isGoal(cur.board)){
        cout << "Solution found with cost = " << cur.g << endl;
        cout << "Moves: " << cur.path << endl;
        cout << "Final board:\n";
        for(auto &row: cur.board){
            for(int val: row) cout << val << " ";
            cout << endl;
        }
        return;
    }

    for(auto &next: getMoves(cur.board)){
        vector<vector<int>> newBoard = next.first;
        if(visited.count(newBoard)) continue;

        int gNew = cur.g + 1;
        int fNew = gNew + manhattan(newBoard);
        pq.push({newBoard, gNew, fNew, cur.path + next.second});
        visited.insert(newBoard);
    }
}
cout << "No solution found!\n";
}

int main(){
    vector<vector<int>> start = {
        {1, 2, 3},
        {4, 8, 0},
        {7, 6, 5}
    };

    solvePuzzle(start);
    return 0;
}

```